UNIVERSITY OF PATRAS

DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATICS AND DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

FPGA Hardware Implementation of waveform segmentation of human ECG signals using LSTM network

Master Thesis

Zintzovas Panagiotis
1096767

MSc
«Hardware and Software Integrated Systems»

Supervisor:
PALIOURAS VASSILIS, Professor

Patras, 07/2025

University of Patras, DEPARTMENT OF COMPUTER ENGINEERING AND INFORMATICS AND DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
Panagiotis Zintzovas
© 2025 - All rights reserved

Examination Committee

Supervisor

Vassilis Paliouras, Professor
Department of Electrical & Computer Engineering

Members

Georgios Theodoridis, Associate Professor
Department of Electrical & Computer Engineering

Nicolas Sklavos, Professor
Department of Computer Engineering & Informatics

ABSTRACT

The purpose of this thesis is the hardware implementation of an LSTM RNN forward pass from a Matlab example ([8]) regarding the waveform segmentation of human electrocardiogram (ECG) signals using an LSTM network. Firstly, we will run the Matlab example and implement a Matlab forward pass that we are going to use as reference. Then we will implement an LSTM forward pass in C++ code and compare our results with its Matlab counterpart. Finally, we will make a hardware implementation of the LSTM forward pass with the use of Vitis HLS, Vivado HLS, and a Zynq™ UltraScale+™ MPSoC ZCU104.

ACKNOWLEDGMENTS

First of all, I would like to express my gratitude and thanks to my supervisor, Prof. Vassilis Paliouras, for his support, guidance, patience, and cooperation. He inspired me to work harder and to continue developing my skills as an M.Sc. student. I would also like to sincerely thank Ioannis Kouretas for his valuable collaboration, insightful feedback, and technical guidance throughout this work. I also thank Prof. Nicolas Sklavos and Prof. Georgios Theodoridis for their participation to the examination committee.

On a more personal note, I would like to thank my friends for their continued support through the last difficult years of my life, special thanks to Thodoris and Aggeliki who hosted me until I found a home when i started, as well as my very dear friend Eirini who has stood with me from the beginning of this journey and has been pushing me to be better and better.

Finally, I want to thank my family, Vassilis, Anna-Maria and Christos, for supporting me all these years of my studies, ‘hey have supported me through thick and thin giving me the strength i need to continue going forward.

KEYWORDS

LSTM: Long-Short-Term Memory.
RNN: Recurrent neural network.
ECG: Electrocardiogram.
HLS: High-Level Synthesis.
QAT: Quantization-Aware Training

CONTENTS

Abstract iv

Acknowledgments vi

Table of Contents ix

List of Figure xi

List Table xii

1 Introduction to LSTM networks 1
- **1.1 Introduction** 1
- **1.2 Basic Structure** 2
- **1.3 Mathematical Equations** 2
  - **1.3.1 Input Gate** 2
  - **1.3.2 Forget Gate** 3
  - **1.3.3 Cell Candidate** 3
  - **1.3.4 Cell State Update** 3
  - **1.3.5 Output Gate and Hidden State** 3
- **1.4 Activation Functions** 3
  - **1.4.1 Sigmoid Function** 3
  - **1.4.2 Hyperbolic Tangent Function** 3
- **1.5 Summary of LSTM Computation** 4
- **1.6 Addional Considerations** 4

2 Introduction to Electrocardiogram signals 6
- **2.1 Basic Information** 6
- **2.2 Single Heart beat Analysis** 6
- **2.3 Analysis of timing features of the electrocardiogram** 7
- **2.4 Summary** 9

3 Waveform Segmentation Using Deep Learning 11
- **3.1 Introduction** 11
- **3.2 Need of segmentation** 11
- **3.3 MATLAB EXAMPLE BREAKDOWN** 12
  - **3.3.1 Raw ECG Input** 12
  - **3.3.2 Network Architecture** 12
  - **3.3.3 Training Options** 12
  - **3.3.4 Training Progress** 13
  - **3.3.5 Testing and Confusion Matrix** 13

4 From matlab To C . . . 16
4.1 C++ Implementation of the LSTM Forward Pass . . . 16
4.2 Public API and Data Structures . . . 16
4.3 Activation and Softmax Primitives . . . 16
4.4 The LSTM Cell (I–F–O–G) . . . 17
4.5 Chunked Ping-Pong Forward Loop . . . 18
4.6 Testbench and Per-Segment Reset . . . 18
4.7 Concluding Remarks . . . 19

5 Quantization Aware Training . . . 20
5.1 Introduction . . . 20
5.2 Background: MATLAB Waveform Segmentation . . . 20
5.3 Why QAT? . . . 20
5.4 QAT Workflow in PyTorch FX . . . 20
5.4.1 Configuring FakeQuant Modules . . . 20
5.4.2 Preparing the Model for QAT . . . 21
5.4.3 Data Loading with Class Balancing . . . 21
5.4.4 Training Loop . . . 22
5.5 Exporting to Vitis HLS . . . 22
5.5.1 Quantization Statistics and Datatype Suggestion . . . 23
5.6 Results . . . 24
5.7 Summary . . . 24

6 Harware Implementation . . . 26
6.1 From C++ to RTL . . . 26
6.1.1 High-Level Synthesis Kernel . . . 26
6.1.2 Fixed-Point Data Types . . . 27
6.1.3 Vitis HLS Analysis and Estimates . . . 28
6.1.4 C/RTL Co-Simulation . . . 31
6.2 Vivado Implementation . . . 32

7 Results and Discussion . . . 34
7.1 FPGA LSTM Forward Pass . . . 34
7.1.1 Host-Side Streaming Loop (Teaser) . . . 34
7.1.2 FPGA Flow and Results . . . 35
7.2 Performance Comparison: MATLAB, C++, QAT and FPGA . . . 37
7.2.1 Discussion . . . 40

8 Future Work . . . 42

9 APPENDIX A . . . 44

10 APPENDIX B . . . 46

BIBLIOGRAPHY . . . 52

LIST FIGURE

Figure 1.1: Repeating module in a standard RNN. Source: [7] 1
Figure 1.2: The repeating module in an LSTM contains four interacting layers. Source: [7] 1
Figure 1.3: LSTM cell. Source: [7] 1

Figure 2.1: Graphical representation of the electrical activity of the heart (ECG). Source [8] 6
Figure 2.2: A single heart beat. [9] 7
Figure 2.3: Normal electrocardiograms recorded from the three standard electrocardiographic leads [2] 8
Figure 2.4: Sinus tachycardia [2] 8
Figure 2.5: Sinus bradycardia [2] 8

Figure 3.1: MATLAB rawNet training-progress plots (10 epochs, piecewise LR). 13
Figure 3.2: Row-normalized confusion chart from MATLAB example. 14

Figure 5.1: Architecture of the MATLAB `rawNet` bidirectional LSTM. 20

Figure 6.1: Dataflow pipeline for `lstm_forward`. 28
Figure 6.2: Function-call / loop hierarchy in HLS. 29
Figure 6.3: Cycle□accurate scheduling of `lstm_forward_chunk`. 29
Figure 6.4: HLS clock-period estimate (6.108 ns). 30
Figure 6.5: Resource utilization for `lstm_forward`. 30
Figure 6.6: Loop-level latency, II and resource breakdown. 31
Figure 6.7: C/RTL Co-Simulation Waveform (showing sample IV handshake). 31
Figure 6.8: Vivado block design showing the custom `lstm_forward` IP, DMA, PS interconnect and reset logic. 32

Figure 7.1: Row-normalized confusion chart from MATLAB example. 37
Figure 7.2: Row-normalized confusion chart from C++ Forward Pass. 37
Figure 7.3: Row-normalized confusion chart from QAT Evaluation. 38
Figure 7.4: Row-normalized confusion chart from FPGA Forward Pass. 39

LIST TABLE

Table 5.1: Comparison of per-class accuracy: MATLAB vs. post-QAT INT-8 (Δ in percentage points) . . . 24

Table 6.1: Post-Route Resource Utilization (Vivado report) . . . 30
Table 6.2: HLS Estimation vs. C/RTL Co-Simulation . . . 31
Table 6.3: Post-Route Resource Utilization (Device: xczu7ev-ffvc1156–2-e) . . . 32
Table 6.4: Power Breakdown (Routed Design) . . . 32

Table 7.1: FPGA LSTM forward-pass throughput . . . 36
Table 7.2: FPGA confusion matrix (row-normalized %) . . . 36
Table 7.3: Per-class accuracy on FPGA (row-normalized %) . . . 36
Table 7.4: Per-class accuracy (%) for MATLAB, C++ (float), QAT (INT-8) and FPGA, with Δ relative to MATLAB. . . . 39

1. INTRODUCTION TO LSTM NETWORKS

1.1 INTRODUCTION

A Long-Short-Term Memory (LSTM) network is a special kind of Recurrent Neural Network (RNN) capable of learning long-term dependencies. They work tremendously well on a wide variety of problems and are now widely used. All RNNs have the form of a chain of repeating modules of a neural network (Fig. 1.1). In standard RNNs, this repeating module will have a very simple structure, such as a tanh layer.

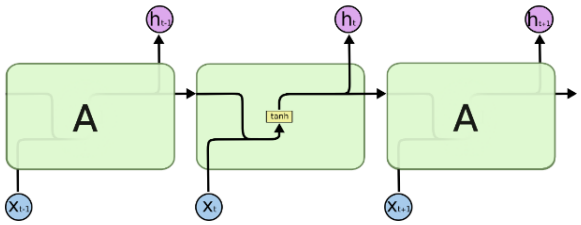


Fig. 1.1: Repeating module in a standard RNN. Source: [7]

LSTM networks also have this *"chain-like"* structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, as we can see in the figure below (Fig. 1.2)

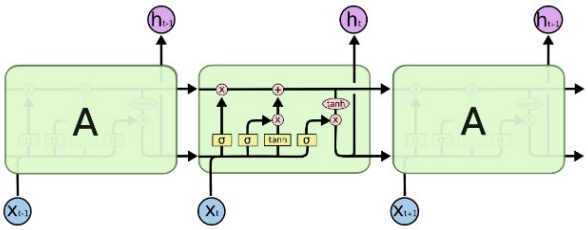


Fig. 1.2: The repeating module in an LSTM contains four interacting layers. Source: [7]

LSTM networks are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn! LSTMs are a kind of recurrent neural network which use the logic of feeding back internal states or output states to the current inputs of neurons. This is done to model sequential events by producing output sequences in which each value at a time t depends on one or more values.

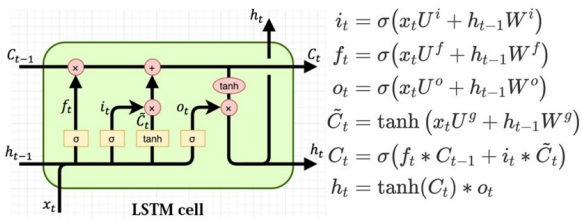


Fig. 1.3: LSTM cell. Source: [7]

The training of an RNN is done through the *"Back Propagation Through Time (BPTT)* algorithm, where a cost function f is minimized with respect to the weights of the neuron according to the equation of the *Gradient Descent* algorithm.

$$\theta^{t+1} = \theta^t - \mu \nabla f(\theta^t) \tag{1.1}$$

Here μ the gradient descent step and θ are the parameters/weights of the neural network. For training, the RNN network unfolds in time, and from there on, corresponding training techniques are followed as in simple non-feedback neural networks.

1.2 BASIC STRUCTURE

An LSTM cell at a time step t processes an input x_t along with the previous hidden state h_{t-1} and the cell state c_{t-1}.

The cell contains several gates:

- **Input gate (i_t)**: controls how much new information flows into the cell.
- **Forget gate (f_t)**: controls how much of the previous cell state is kept.
- **Output gate (o_t)**: controls how much of the cell state is exposed to the hidden state.
- **Cell candidate($\tilde{\mathbf{c}}_t$)**: a candidate for updating the cell state.

1.3 MATHEMATICAL EQUATIONS

At each time step t, the LSTM cell computes the following:

1. **Input Gate**
2. **Forget Gate**
3. **Cell Candidate**
4. **Cell State Update**
5. **Output Gate and Hidden State**

1.3.1 Input Gate

: The input gate decides which new information should be added to the cell state.

$$i_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) \tag{1.2}$$

Here,

- $\sigma(\cdot)$ is the logistic sigmoid function,
- W_i is the weight matrix for the input,
- U_i is the recurrent weight matrix, and
- b_i is the bias vector

1.3.2 Forget Gate

The forget gate decides what information to discard from the previous cell state.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) \tag{1.3}$$

1.3.3 Cell Candidate

A candidate value for updating the cell state is computed using a hyperbolic tangent function.

$$\tilde{\mathbf{c}}_t = \tanh\Big(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + b_c\Big)$$

1.3.4 Cell State Update

The new cell state is computed by combining the scaled previous cell state with the candidate value:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

1.3.5 Output Gate and Hidden State

Finally, the output gate determines what part of the cell state will be output as the hidden state:

$$\mathbf{o}_t = \sigma\Big(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + b_o\Big)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

1.4 ACTIVATION FUNCTIONS

1.4.1 Sigmoid Function

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

which maps any real number z into the interval $[0, 1]$.

1.4.2 Hyperbolic Tangent Function

The hyperbolic tangent function is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

which maps z into the interval $[-1, 1]$.

1.5 SUMMARY OF LSTM COMPUTATION

At each time step t, the LSTM cell performs the following computations:

$$
\begin{aligned}
\mathbf{i}_t &= \sigma\Big(W_i\mathbf{x}_t + U_i\mathbf{h}_{t-1} + b_i\Big) \\
\mathbf{f}_t &= \sigma\Big(W_f\mathbf{x}_t + U_f\mathbf{h}_{t-1} + b_f\Big) \\
\tilde{\mathbf{c}}_t &= \tanh\Big(W_c\mathbf{x}_t + U_c\mathbf{h}_{t-1} + b_c\Big) \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\
\mathbf{o}_t &= \sigma\Big(W_o\mathbf{x}_t + U_o\mathbf{h}_{t-1} + b_o\Big) \\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)
\end{aligned}
$$

1.6 ADDIONAL CONSIDERATIONS

- **Training**: During training, all these weights (W_i,U_i,b_i,etc.) are learned so that the network can capture temporal dependencies in the data.
- **Inference**: During inference, the LSTM cell is executed for each time step in the sequence, updated its internal cell state and hidden state accordingly.
- **Implementation in Hardware**: When implementing an LSTM on FPGA, you typically convert these floating-point operations into fixed-point arithmetic (using approximations for the activation functions and careful quantization of weights and states). This requires a detailed analysis of dynamic range and precision, and often involves techniques like lookup tables (LUTs) for nonlinear functions.

2. INTRODUCTION TO ELECTROCARDIOGRAM SIGNALS

2.1 BASIC INFORMATION

In this chapter, we will cover some basic information about human ECG signals. Electrocardiogram (ECG) signals represent the activity of the human heart and consist of several waveforms [1]. These waveforms can be divided into 3 beat morphologies [4]:

- P wave — A small deflection before the QRS complex representing atrial depolarization
- QRS complex — Largest-amplitude portion of the heartbeat
- T wave — A small deflection after the QRS complex representing ventricular re-polarization

The duration and shape of each waveform and the distances between different peaks are used to diagnose heart disease and check the proper function of the human heart.

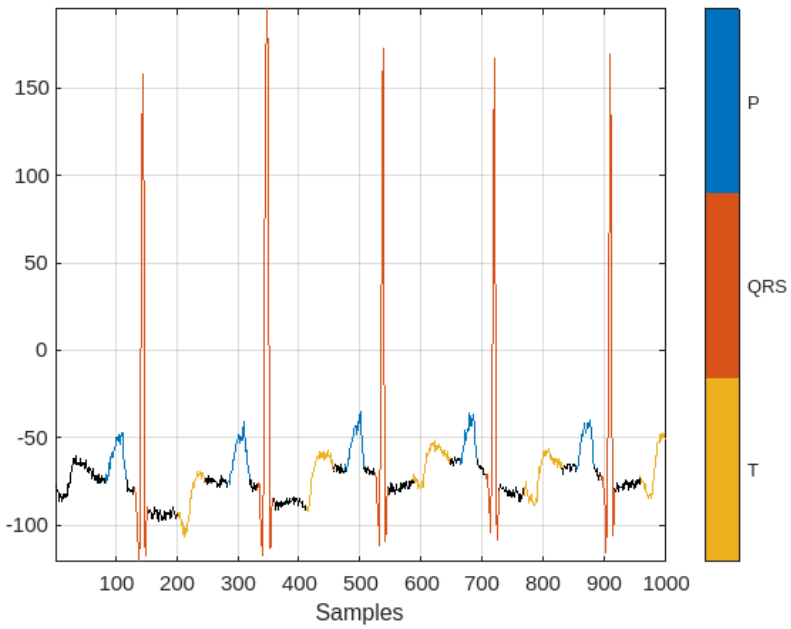


Fig. 2.1: Graphical representation of the electrical activity of the heart (ECG). Source [8]

2.2 SINGLE HEART BEAT ANALYSIS

In Figure 2.1, we can see a graphical representation of the electrical activity of the heart. In the following figure 2.2, we will take a closer look at the 3 beat morphologies that take place in the process of a single heart beat.

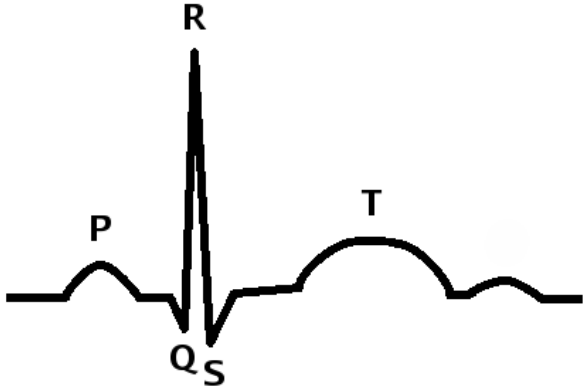


Fig. 2.2: A single heart beat. [9]

Before diving into the human heart beat process, we must first briefly mention the existence and function of the sinoatrial (SA) and atrioventricular (AV) nodes. The sinoatrial (SA) node represents a cluster of myocytes with peacemaker activity. Under normal circumstances, it generates electrical impulses that set the rhythm of the heart[6]. The atrioventricular (AV) node is a small structure in the heart [5]. Its purpose is to connect the electrical systems of the atria and the ventricles, providing electrical impedance from the atria and an intrinsic pacemaker in its absence. A single heart beat begins when an electrical signal from the SA node in the atria propagates throughout both atria[3]. This signal depolarizes the atria, therefore causing them to contract and force blood into the ventricles. On the ECG, this is seen as the P wave. The ventricles follow the same fashion when an electrical signal from the AV node depolarizes both ventricles, causing them to contract. This electrical activity is displayed in the ECG as the QRS complex. Finally, the ventricles return to their previous state and prepare for the next contraction, resulting in the repolarization wave, the T wave.

2.3 ANALYSIS OF TIMING FEATURES OF THE ELECTROCARDIOGRAM

Time intervals between the different waves of the ECG can help doctors diagnose cardiac abnormalities. The time between heart beats is one of the most basic qualities that can be extracted from the ECG 2.3.

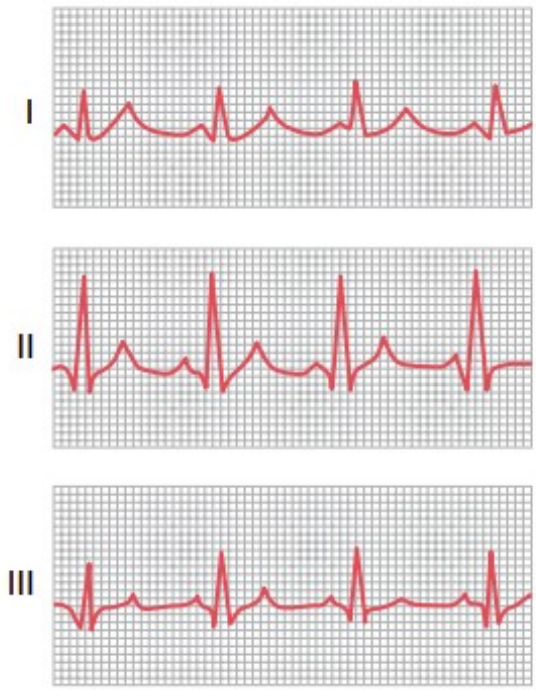


Fig. 2.3: Normal electrocardiograms recorded from the three standard electrocardiographic leads [2]

Heart malfunction due to an abnormal rhythm of the heart is called an arrhythmia. A fast heart rate, called tachycardia, generally results from an increased body temperature, stimulation of the heart by the sympathetic nerves, or toxic conditions of the heart 2.4 .

Fig. 2.4: Sinus tachycardia [2]

The opposite condition, slow heart rate or bradycardia, may result from problems with the heart's electrical system and natural pacemaker 2.5.Extreme cases of bradycardia may mean that the heart is not pumping enough blood to meet the body's needs.

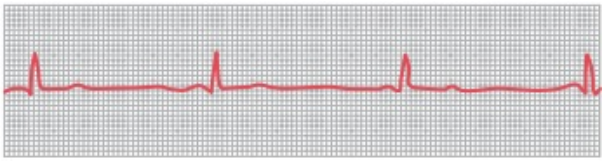


Fig. 2.5: Sinus bradycardia [2]

To aid in the diagnosis of such conditions, the ECG is processed to extract the average heart rate of the patient and display it continuously at the base station. The duration of the QRS complex is also calculated and displayed at the base station. Since the QRS complex lasts as long as depolarization of the ventricles lasts, an abnormally

prolonged or otherwise irregular duration of the complex indicates improper electrical conduction throughout the heart. This can be due to the deconstruction of heart muscle throughout the ventricular system that is then replaced with scar tissue, or the appearance of multiple small local blocks in the Purkinje system, the part of the heart that conducts electrical impulses to the ventricles. A normal QRS complex lasts 0.06 to 0.08 seconds; anything longer than 0.09 seconds is considered abnormal.

2.4 SUMMARY

The essence of what we mentioned is that ECG's are primarily used to monitor and assess the electrical activity of the heart. Some of the most important applications of ECG's are:

- **Diagnosis of Heart Conditions:** ECGs help detect abnormalities such as arrhythmias, myocardial infarction (heart attack), and other cardiac disorders.
- **Monitoring Cardiac Health:** In hospitals and clinics, ECG's are used for continuous monitoring of heart activity, especially in high-risk patients or during surgeries.
- **Evaluating Treatment Effectiveness**: Doctors use ECG data to assess the effectiveness of medications or interventions designed to improve heart function.
- **Research and Development:** In biomedical research, ECG's provide critical data for the development of advanced diagnostic tools, the training of machine learning algorithms, and understanding cardiac physiology.
- **Preventative Health Screenings:** Regular ECG screenings can help identify early signs of heart problems, allowing for timely intervention.

But, just like every other signal, ECG's are susceptible to noise and other factors which can produce spikes in ECG signals making it harder to ensure accurate ECG analysis. Body movement, breathing, electrode movement on the body and power line inferences are a few examples that can affect the process of an electrocardiogram examination.

3. WAVEFORM SEGMENTATION USING DEEP LEARNING

3.1 INTRODUCTION

This master thesis builds upon the methodology presented in the MATLAB example *'Waveform Segmentation Using Deep Learning'* provided my MathWorks [4]. The example demonstrates how to segment ECG waveforms into key regions, such as the 3 beat morphologies we analyzed in the previous chapter on page 6 (P wave, QRS complex, T wave), using deep learning techniques. Based on this approach, our work adapts and extends the segmentation strategy to develop an optimized framework for ECG signal analysis on FPGA hardware. Using insights and techniques from the MATLAB example, our aim is to improve real-time signal processing performance while maintaining the diagnostic integrity of the segmented ECG data.

3.2 NEED OF SEGMENTATION

According to the MATLAB example, segmenting an ECG waveform into its 3 main regions - typically corresponding to the P wave, the QRS complex, and the T wave - is useful for several reasons:

1. **Isolating Distinct Cardiac Phases**:
   - **P wave:** Indicates atrial depolarization.
   - **QRS complex:** Reflects ventricular repolarization.
   - **T wave:** Represents ventricular repolarization.
     By segmenting these phases, you can focus on the unique characteristics of each phase, which are critical for identifying normal and abnormal cardiac behaviors.
2. **Enhanced Feature Extraction:**
   When you segment the ECG signal, you can extract more meaningful features from each individual region. This targeted analysis allows for improved detection of subtle changes in the waveform, which might be indicative of various cardiac conditions.
3. **Improved Classification and Diagnosis:**
   For machine-learning or deep-learning applications, having clearly segmented regions helps the algorithm learn and distinguish between the different components of the ECG signal. This can lead to more accurate classification of heartbeats or detection of anomalies, as each segment can be analyzed separately and then recombined for a holistic diagnosis.
4. **Data Normalization and Consistency:**
   Segmentation facilitates aligning the ECG waveforms across different heartbeats. This consistency is important when training algorithms because it reduces variability that might otherwise obscure the underlying patterns associated with specific cardiac events.

In summary, by segmenting the ECG waveform into its component parts, the MATLAB example demonstrates how to isolate and analyze the critical phases of the cardiac cycle more effectively, leading to better feature extraction and ultimately to a more reliable diagnosis through deep learning techniques.

3.3 MATLAB EXAMPLE BREAKDOWN

In this section we step through the MathWorks "Waveform Segmentation Using Deep Learning" rawNet example in detail.

3.3.1 Raw ECG Input

The network is trained directly on 1D ECG segments of length 5000 samples. MATLAB windows the 210 recordings into 2835 non-overlapping segments (250 Hz sample rate), then uses all in-memory data for faster training:

```
tallTrainSet = tall(trainDs);
tallTestSet  = tall(testDs);

trainData = gather(tallTrainSet);   % [2835×2 cell]: {signal,labels}
testData  = gather(tallTestSet);
```

Listing 1: Create tall arrays and gather into workspace

3.3.2 Network Architecture

Define a single-layer LSTM (200 hidden units) that outputs a class per time-step:

```
layers = [ ...
    sequenceInputLayer(1)
    lstmLayer(200,'OutputMode','sequence')
    fullyConnectedLayer(4)
    softmaxLayer];
```

Listing 2: rawNet layer definition

3.3.3 Training Options

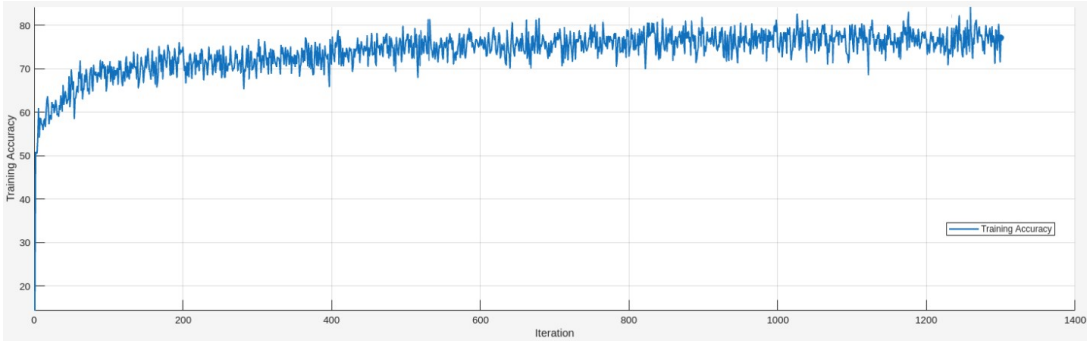
They use ADAM with an initial LR of 0.01, dropping by 0.1 every 3 epochs, for 10 total epochs:

```
options = trainingOptions('adam', ...
    'InitialLearnRate',0.01, ...
    'MaxEpochs',10, ...
    'MiniBatchSize',50, ...
    'LearnRateSchedule','piecewise', ...
    'LearnRateDropFactor',0.1, ...
    'LearnRateDropPeriod',3, ...
    'GradientThreshold',1, ...
    'Plots','training-progress', ...
    'Verbose',false);
```

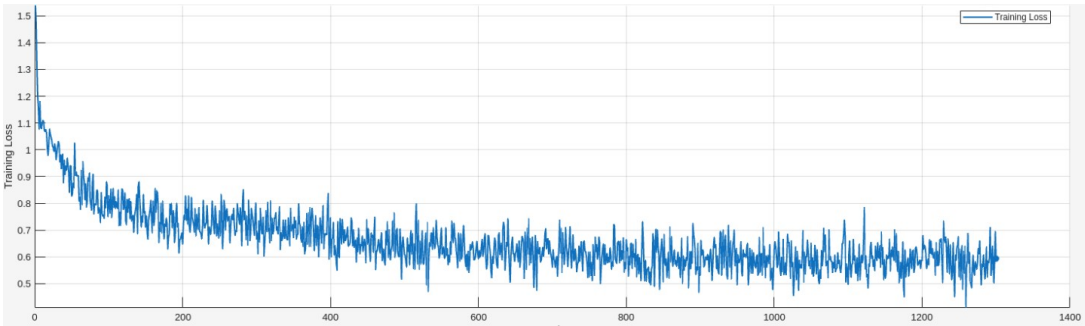
Listing 3: TrainingOptionsADAM settings

3.3.4 Training Progress

Training reaches ≈77% sample-wise accuracy by epoch 10, as shown in the MATLAB "training-progress" plots:



(a) Training Accuracy over iterations



(b) Training Loss over iterations

Fig. 3.1: MATLAB rawNet training-progress plots (10 epochs, piecewise LR).

3.3.5 Testing and Confusion Matrix

Finally, they classify the 30% test split with 'minibatchpredict' and plot a row-normalized confusion chart:

```
scores   = minibatchpredict(rawNet, testData(:,1), ...
            'MiniBatchSize',50,'UniformOutput',false);
predTest = scores2label(scores, categories(trainData{1,2}));

confusionchart(...
  vertcat(testData{:,2}), ...
  vertcat(predTest{:}), ...
  'Normalization','row-normalized');
```

Listing 4: Test prediction and confusion chart

The resulting confusion matrix (Fig. 3.2) shows per-class accuracies of approximately 86.9% (n/a), 39.4% (P), 60.9% (QRS), and 57.9% (T).

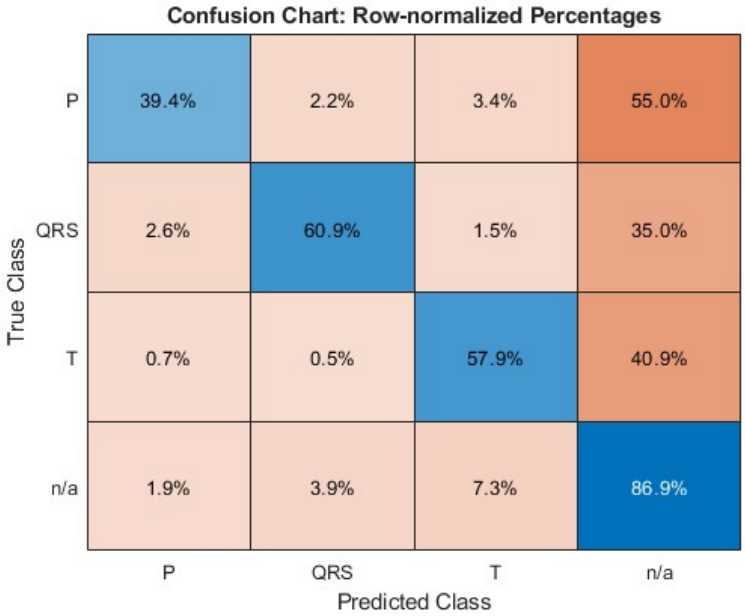


Fig. 3.2: Row-normalized confusion chart from MATLAB example.

4. FROM MATLAB TO C

4.1 C++ IMPLEMENTATION OF THE LSTM FORWARD PASS

In this section we show how we took the MATLAB example and recast its LSTM forward-pass into stand-alone, idiomatic C++. Our goal was a one-to-one mapping of functionality, but using only STL containers and `<cmath>`.

4.2 PUBLIC API AND DATA STRUCTURES

All of our dimensions and weights live in two files:

- `lstm_forward.h` declares

```
constexpr int INPUT_SIZE  = 1;
constexpr int HIDDEN_SIZE = 200;
constexpr int OUTPUT_SIZE = 4;

// pulls in all gate weights:
// Wi\_input, Wi\_recurrent, Wi\_bias, Wf\_*, Wc\_*, Wo\_*,
// plus final W\_fc[4][200] and b\_fc[4]
#include "lstm_weights.h"

void lstm_forward(
  const std::vector<float>& ecg,
  std::vector<int>&         pred,
  int                       numSamples,
  int                       chunkSize
);
```

- `lstm_weights.h` holds static `float` arrays of size `[200][200]` or `[4][200]`.

4.3 ACTIVATION AND SOFTMAX PRIMITIVES

We implement the non-linearities simply:

```
// logistic sigmoid for I, F, O gates
static float sigmoid_fast(float x) {
  return 1.0f/(1.0f + std::exp(-x));
}
// tanh for the cell candidate and hidden update
static float tanh_fast(float x) {
  return std::tanh(x);
}
```

```
// row-stable softmax + argmax
static void softmax(const float in[OUTPUT_SIZE],
                    float       out[OUTPUT_SIZE]) {
  float m = *std::max_element(in,in+OUTPUT_SIZE);
  float sum = 0.f;
  for(int i=0;i<OUTPUT_SIZE;++i){
    out[i] = std::exp(in[i]-m);
    sum += out[i];
  }
  for(int i=0;i<OUTPUT_SIZE;++i) out[i] /= (sum>0?sum:1);
}
static int argmax(const float p[OUTPUT_SIZE]) {
  return int(std::distance(p,std::max_element(p,p+OUTPUT_SIZE)));
}
```

4.4 THE LSTM CELL (I–F–O–G)

Here's the heart of the network: one loop over `HIDDEN_SIZE` that computes input, forget, output gates and the cell candidate, then updates state.

```
static void lstm_cell(
  const float  x_in[INPUT_SIZE],
  const float  h_i[HIDDEN_SIZE],
  const float  c_i[HIDDEN_SIZE],
        float  h_o[HIDDEN_SIZE],
        float  c_o[HIDDEN_SIZE])
{
  float x = x_in[0];
  for(int i=0;i<HIDDEN_SIZE;++i){
    // 1) I-gate
    float gi = Wi_bias[i] + Wi_input[i][0]*x;
    for(int j=0;j<HIDDEN_SIZE;++j) gi += Wi_recurrent[i][j]*h_i[j];
    gi = sigmoid_fast(gi);

    // 2) F-gate (same pattern)
    float gf = /* ... */;  gf = sigmoid_fast(gf);

    // 3) O-gate
    float go = /* ... */;  go = sigmoid_fast(go);

    // 4) cell candidate G
    float gg = /* ... */;  gg = tanh_fast(gg);

    // 5) state update
    float nc = gf*c_i[i] + gi*gg;
    c_o[i] = nc;
    h_o[i] = go * tanh_fast(nc);
  }
}
```

4.5 CHUNKED PING-PONG FORWARD LOOP

To mirror streaming hardware, we alternate two state buffers:

```
void lstm_forward(const std::vector<float>& ecg,
                  std::vector<int>&         pred,
                  int                       numSamples,
                  int                       chunkSize)
{
  pred.resize(numSamples);
  std::vector<float> h0(HIDDEN_SIZE,0), c0(HIDDEN_SIZE,0);
  std::vector<float> h1(HIDDEN_SIZE,0), c1(HIDDEN_SIZE,0);

  int done = 0;
  while(done < numSamples){
    int thisC = std::min(chunkSize, numSamples-done);
    for(int t=0;t<thisC;++t){
      float x[1] = { ecg[done+t] };
      float logits[OUTPUT_SIZE];
      if(((done+t)&1)==0){
        lstm_cell(x, h0.data(), c0.data(), h1.data(), c1.data());
        softmax_readout(h1.data(),logits);
      } else {
        lstm_cell(x, h1.data(), c1.data(), h0.data(), c0.data());
        softmax_readout(h0.data(),logits);
      }
      pred[done+t] = argmax(logits);
    }
    done += thisC;
  }
}
```

4.6 TESTBENCH AND PER-SEGMENT RESET

Finally, in `testbench.cpp` we load the ecg signals that Matlab used for testing into `vector<float>`, then:

1. Loop over each segment of length 5000
2. `slice` the input vector to one segment
3. Call `lstm_forward(slice, pred_seg, 5000, 5000)` to zero the hidden/cell state at each segment start
4. Append `pred_seg` to a master `all_pred`
5. Compute confusion matrix and accuracy

```
for(int seg=0; seg<N_SEGMENTS; ++seg){
  auto begin = ecg.begin() + seg*SEGMENT_LEN;
  vector<float> slice(begin, begin+SEGMENT_LEN);
  vector<int>   pseg;
  lstm_forward(slice, pseg, SEGMENT_LEN, CHUNK_SIZE);
  all_pred.insert(all_pred.end(), pseg.begin(), pseg.end());
}
```

4.7 CONCLUDING REMARKS

- We chose single-precision floats to balance performance and MATLAB-compatible accuracy.
- The I–F–O–G ordering and ping-pong buffers give a clear, in-place streaming design.
- Chunking allows you to reset the state per segment, exactly matching MATLAB's behavior.
- We print the results in a similar way as in the Matlab example as a confusion matrix.

In Chapter 5 we will prepare our weights and ecg signal data for fixed point representation.

5. QUANTIZATION AWARE TRAINING

5.1 INTRODUCTION

In this chapter, we describe how we extended the MATLAB waveform segmentation example[1] to an FPGA flow. Since the original ‘rawNet‘ network uses full-precision floats, we applied *Quantization-Aware Training* (QAT) in PyTorch FX to produce an 8-bit–quantized model suitable for synthesis with Vitis HLS targeting the Zynq™ UltraScale+™ MPSoC ZCU104 kit.

5.2 BACKGROUND: MATLAB WAVEFORM SEGMENTATION

The MATLAB example implements a bidirectional LSTM that takes 5000-sample ECG segments and outputs a per-sample class (P, QRS, T, or none). Figure 5.1 illustrates its architecture.

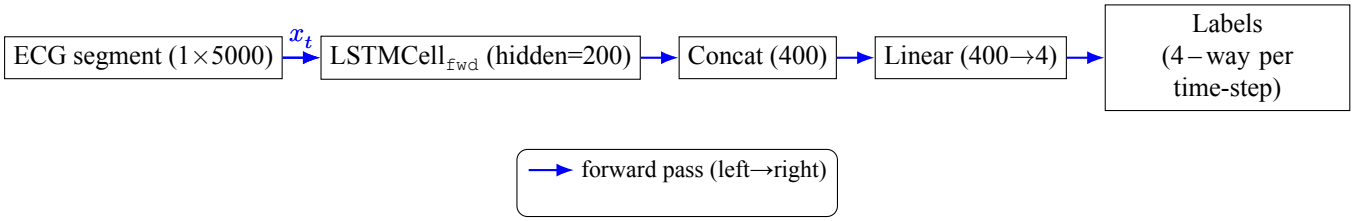


Fig. 5.1: Architecture of the MATLAB `rawNet` bidirectional LSTM.

5.3 WHY QAT?

Floating-point networks consume excessive resources on FPGA. By simulating quantization effects during training, QAT lets us:

- Maintain accuracy close to the original full-precision model.
- Export int8 weights, int32 biases and activation parameters.
- Seamlessly integrate into Vitis HLS C++ for hardware synthesis.

5.4 QAT WORKFLOW IN PYTORCH FX

5.4.1 Configuring FakeQuant Modules

We use symmetric per-tensor quantization for weights and affine for activations. Listing 5 shows our FX QConfig.

[1] `https://mathworks.com/help/signal/ug/waveform-segmentation-using-deep-learning.html#WaveformSegmentationUsingDeepLearningExample-3`

```python
from torch.ao.quantization import (
    FakeQuantize,
    get_default_qconfig,
    QConfig
)

# 8-bit symmetric weight quantization (-127…127)
W_FQ = FakeQuantize.with_args(
    observer=get_default_qconfig("qnnpack").weight().observer,
    quant_min=-127, quant_max=127,
    dtype=torch.qint8
)

# 8-bit affine activation quantization (0…255)
A_FQ = FakeQuantize.with_args(
    observer=get_default_qconfig("qnnpack").activation().observer,
    quant_min=0, quant_max=255,
    dtype=torch.quint8
)

qconfig_fx = QConfig(activation=A_FQ, weight=W_FQ)
```

Listing 5: FX QConfig for weight and activation quantization

5.4.2 Preparing the Model for QAT

We insert observers and fake-quant nodes via FX:

```python
from torch.ao.quantization.fx import (
    prepare_qat_fx,
    convert_fx
)

# attach qconfig and prepare
model.qconfig = qconfig_fx
prepare_qat_fx(model, inplace=True)

# run a forward pass on a few batches to calibrate observers
for x, _ in calib_loader:
    model(x)
```

Listing 6: Inserting QAT modules and running a calibration pass

5.4.3 Data Loading with Class Balancing

To mitigate the heavy class imbalance in our ECG segments (lots of "n/a" vs. fewer P/QRS/T), we replace the vanilla random sampler with a `WeightedRandomSampler`.

```python
# in make_train_loader()
frac_nna = [(np.bincount(y, minlength=4)[0] / y.size) for y in Y]
weights  = 1.0 + 2.0*(1.0 - np.array(frac_nna))
```

```
sampler = WeightedRandomSampler(weights, len(weights), replacement=True)
```

We compute, for each segment i,

$$w_i = 1 + 2\,(1 - f_{n/a}^{(i)}),$$

where $f_{n/a}^{(i)}$ is the fraction of "n/a" labels in segment i. This gives higher sampling weight to segments with more P/QRS/T events, improving per-class learning.

5.4.4 Training Loop

Our QAT loop (Listing 7) runs for 10 epochs, halving the LR at epoch 5.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=5, gamma=0.5
)

for epoch in range(10):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits.view(-1,4), y.view(-1))
        loss.backward()
        optimizer.step()
    scheduler.step()
    print(f"Epoch {epoch}: loss {loss.item():.4f}")
# convert to int8
model_int8 = convert_fx(model)
```

Listing 7: Quantization-Aware Training loop

5.5 EXPORTING TO VITIS HLS

Once QAT and the INT-8 checkpoint are ready, our script produces everything the HLS project needs:

- **Fixed-point weight / bias headers:**
  - `W_all_fixed.h`: fused LSTM input+recurrent weights (ap_fixed<16,6> literals)
  - `B_all_fixed.h`: fused LSTM biases
  - `W_fc_fixed.h`: final FC weights
  - `b_fc_fixed.h`: final FC biases
- **Activation LUT headers (Q2.6):**
  - `sigmoid_lut.h`
  - `tanh_lut.h`
  - `exp_lut.h`

- **Testbench and wrapper:**
  - `testbench.cpp`: drives `ecg_testData_int8.bin` through the HLS kernel and compares to ground-truth.
  - `lstm_forward.h/cpp`: the generated C++ RTL interface for the LSTM cell.
- **Quantization statistics:**
  - `act_ranges.json`: recorded min/max for every observer
  - `qat_stats.json`: gate-state μ/σ and percentiles
  - `datatype_suggestion.txt`: recommended `ap_fixed<N,I>` for state_t
- **Binary test vectors:**
  - `ecg_testData_int8.bin`: raw uint8 ECG segments after QAT quantization

All of these files are written into the HLS include directory (see Appendix **??**). The HLS project simply `#includes`:

```
#include "W_all_fixed.h"
#include "B_all_fixed.h"
#include "W_fc_fixed.h"
#include "b_fc_fixed.h"
#include "sigmoid_lut.h"
#include "tanh_lut.h"
#include "exp_lut.h"
#include "lstm_forward.h"
```

and then in `testbench.cpp` reads `ecg_testData_int8.bin` to drive the kernel and report a pass/fail against the MATLAB gold labels.

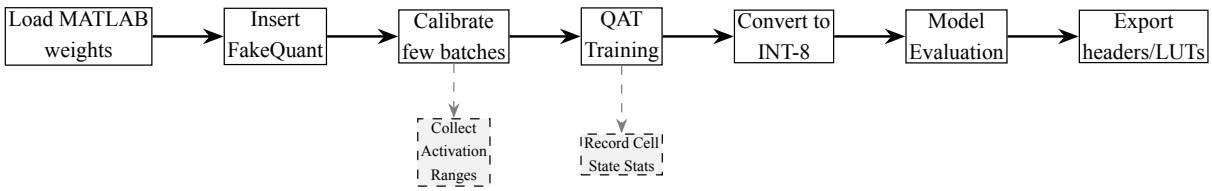
This complete export pipeline ensures that the exact same quantized parameters and activation ranges measured during QAT are used in the FPGA flow without any manual conversion.

5.5.1 Quantization Statistics and Datatype Suggestion

During QAT we register hooks to capture:

- **Max gate activation** $g_{\max}$ across the network,
- **Cell-state distribution** (mean, standard deviation, 0.5 % and 99.5 % percentiles),
- **Suggested fixed-point format** based on worst-case weight sum.

We can see the process in the picture below:



For instance, our script printed:

```
   Suggested state_t : ap_fixed<16,6>
    - weight head-room ×1.25
    - MSB utilisation  80.3%
  =0.002,  =0.085, 5 0.425
 p0.5=-0.310, p99.5=+0.312
```

We then used that to set

```
typedef ap_fixed<16,6> state_t;
```

in our HLS headers.

5.6 RESULTS

Table 5.1 compares the full-precision and int8 models on the test set:

Table 5.1: Comparison of per-class accuracy: MATLAB vs. post-QAT INT-8 (Δ in percentage points)

Class	MATLAB (%)	INT-8 QAT (%)	Δ (INT-8–MATLAB) (%)
n/a	86.9%	86.68%	–0.22%
P	39.4%	40.25%	+0.85%
QRS	60.9%	61.00%	+0.10%
T	57.9%	58.38%	+0.48%

5.7 SUMMARY

After applying Quantization-Aware Training, the per-class accuracies changed as follows (Δ = QAT – MATLAB):

- **n/a:** decreased by –0.22 pp (from 86.9% to 86.68%)
- **P:** increased by +0.85 pp (from 39.4% to 40.25%)
- **QRS:** increased by +0.10 pp (from 60.9% to 61.00%)
- **T:** increased by +0.48 pp (from 57.9% to 58.38%)

Compared to the MATLAB floating-point baseline, our fully INT-8 model exhibits a modest redistribution of accuracy across classes. The majority "n/a" class dips slightly (–0.22 pp) as the network becomes more sensitive to real waveform events, while all three true-event classes gain: P rises by +0.85 pp, QRS by +0.10 pp, and T by +0.48 pp. This shift reflects our 30-epoch QAT training as well as the weighted-sampler strategy, which over-samples segments rich in P/QRS/T patterns, trading a bit of background accuracy for stronger overall detection of true waveform features.

In Chapter 5 we describe how these quantized headers and test vectors are integrated into our HLS C++ design and synthesized on the Zynq™ UltraScale+™ MPSoC ZCU104 board.

6. HARWARE IMPLEMENTATION

6.1 FROM C++ TO RTL

The next big step is moving from software to hardware. The Quantization Aware Training gave us the tools we need to start transforming our high level C++ code into a version that can be synthesized and converted into a register-transfer level (RTL) hardware description language (HDL) like Verilog or VHDL. As mentioned before, we will be using Vitis HLS (2023.1) as our High-Level Synthesis tool. Having prepared our fixed-point weight and activation tables via QAT, we now feed our C++ forward-pass implementation into Vitis HLS 2023.1. Vitis HLS translates the high-level LSTM kernel into cycle-accurate RTL, applies user directives to guide pipelining and dataflow, and produces both synthesis estimates and an IP that we can later drop into Vivado.

6.1.1 High-Level Synthesis Kernel

Listing 8 summarizes the top-level interface and key design points in our HLS C++:

```
/**
 * @file   lstm_forward.cpp
 * @brief  High-level synthesis of an 8-bit LSTM segmentation kernel.
 *
 * - Reads raw ECG samples via AXI-Stream.
 * - Processes in fixed-point (state_t: ap_fixed<16,6>).
 * - Unrolls 5000-sample segments with II=1 pipelined loop.
 * - Uses ping-pong buffers for hidden/cell state.
 * - Exports per-sample predictions via AXI-Stream.
 *
 * Functions:
 *   lstm_forward()              Top-level kernel w/ AXI interfaces.
 *   lstm_forward_chunk()        Dataflow wrapper for chunked processing.
 *   TIME_LOOP()                 Inner loop over time-steps.
 *   lstm_cell()                 Compute one LSTM cell update.
 *   fully_connected_and_softmax Compute logits + softmax output.
 *   sigmoid_fast(), tanh_fast() Lookup + interpolate in partitioned LUTs.
 *   softmax(), argmax()         Row-stable softmax + label select.
 *
 * HLS Pragmas:
 *   INTERFACE axis     register both port=…
 *   INTERFACE s_axilite bundle=control
 *   BIND_STORAGE for weights → ROM in BRAM
 *   ARRAY_PARTITION for LUTs & state buffers
 *   DATAFLOW on the chunk wrapper
 *   PIPELINE II=1 on TIME_LOOP
 */
```

Listing 8: Top-level HLS kernel overview and pragmas

Below is a walk-through of how data flows through the kernel:

- **lstm_forward(...):** - AXI-Stream input/output ports (axis and s_axilite). - Orchestrates bursts of size `chunkSize`, zeroing state at each segment boundary.
- **lstm_forward_chunk(...):** - Annotated with `#pragma HLS DATAFLOW` to overlap DMA read/write with compute.
- **TIME_LOOP:** - Simple for-loop over each time-step $t \in [0, \dots, \text{chunkSize} - 1]$. - Drives two ping-pong state buffers so each call can read "old" state while writing "new".
- **lstm_cell(...):** - Inlined via (`#pragma HLS INLINE`). - Executes four gate computations (Input, Forget, Candidate, Output) using MAC loops. - Activation functions use partitioned LUTs for single-cycle lookup.
- **fully_connected_and_softmax(...):** - Dense read-out of hidden vector + row-stable softmax + `argmax` label.
- **Activation LUTs and State Buffers:** – Both are `#pragma HLS ARRAY_PARTITION`ed to give every lookup or read/write a separate hardware port, ensuring single-cycle access.

All of these design choices, loop pipelining, dataflow, array partitioning, and storage binding, come together to give us a fully pipelined, one-sample-per-clock LSTM engine whose timing, area, and throughput we can now push into Vivado for RTL co-simulation, place & route, and bitstream generation.

6.1.2 Fixed-Point Data Types

In this section, we will demonstrate the benefits of our exhaustive analysis of the ecg signals and rawNet weights parameters during QAT. By thorough examination of the QAT process, the adjustment of the full-precision floats to quantized values and the results of our "hooks" (5.5.1) we managed to get a clear picture of the data width we need to avoid overflow and saturation.

```
typedef ap_int<8>       input_t;   // signed two's-complement 8-bit ECG samples

// hidden-state & cell-state: 16 total bits, 6 integer → 10 fractional
// matches our suggested ap_fixed<16,6> from QAT statistics
typedef ap_fixed<16,6>  state_t;

// accumulator for 200-tap MAC: needs ~log2(200 max(|w| |x|))   32 bits
// we give 24 int bits + 8 frac bits → 32 total
typedef ap_fixed<32,24> acc_t;

// since cell update sums gf·c + gi·ĝ but stays near same dynamic range,
// we narrow to 26 total bits to save area while avoiding saturation
typedef ap_fixed<26,14> cell_t;

// Softmax exponentials & sum: needs extra fractional precision
typedef ap_fixed<24,6>  sum_t;

// AXI-Stream packet: 8-bit data, no side-channels
typedef ap_axiu<8,0,0,0> in_pkt_t;
typedef ap_axiu<8,0,0,0> out_pkt_t;  // prediction label packed into 8 bits
```

Width selection rationale:

- `input_t` (`ap_int<8>`) holds signed two's-complement ECG samples as produced by our QAT export.
- `state_t` follows the QAT-suggested `ap_fixed<16,6>` based on gate-state μ/σ and worst-case weight sums. We added a few extra bits to improve accuracy since we can afford it.
- `acc_t` must accumulate up to 200 products: $\log_2(200) \approx 8$ bits headroom plus the dynamic range of `state_t` → 24 int + 8 frac bits.
- `cell_t` provides a bit more headroom (2 extra int bits, 4 extra frac bits) for the sum $\mathtt{gf} \cdot c + \mathtt{gi} \cdot \tilde{c}$.
- `sum_t` holds exponentials and running sums in softmax-6 int bits for the largest `exp` plus 18 frac bits for precision.
- `in_pkt_t`/`out_pkt_t` simply wrap the 8-bit sample or label into an AXI-Stream word.

6.1.3 Vitis HLS Analysis and Estimates

We used Vitis HLS to get a full picture of our kernel's performance, pipelining and area. The key takeaways are:

1. **Dataflow Concurrency**
   The Dataflow Viewer (Fig. 6.1) shows three stages (`unpack_axis`, `compute`, `pack_axis`) running in a streaming fashion so that once the pipeline is full we issue one sample per cycle even though the top□level `TIME_LOOP` itself isn't explicitly pipelined end-to-end.

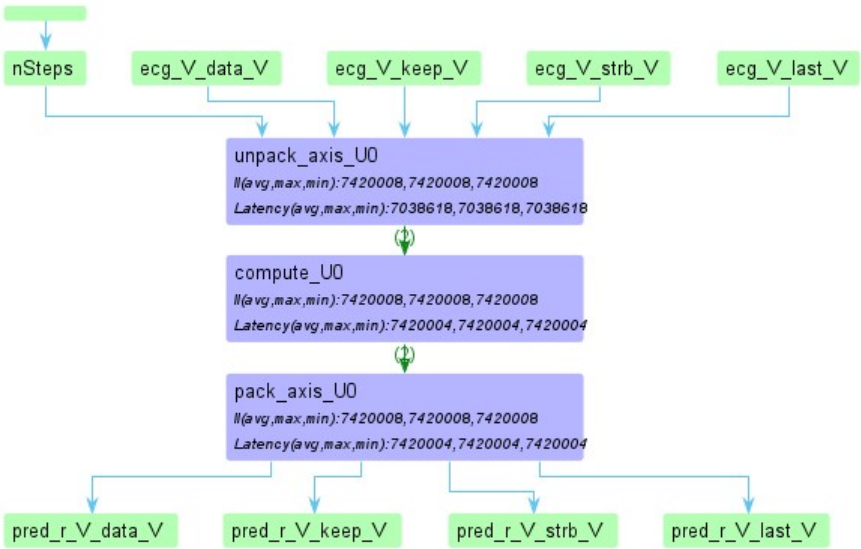


Fig. 6.1: Dataflow pipeline for `lstm_forward`.

2. **Module & Loop Hierarchy**
   The "Modules & Loops" graph (Fig. 6.2) breaks the design into `BURST_LOOP`, `lstm_forward_chunk`, and the three engines (`unpack_axis`, `compute`, `pack_axis`). Under `compute` you see `TIME_LOOP` → `lstm_cell` (with its MAC and UPD loops) → the softmax loops, exactly mirroring our C++ structure.

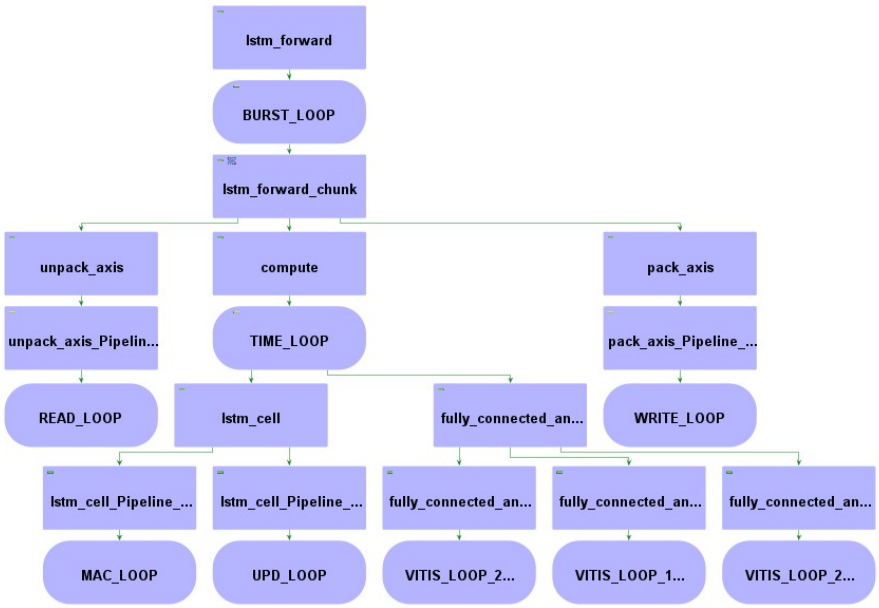


Fig. 6.2: Function-call / loop hierarchy in HLS.

3. **Cycle-Accurate Schedule**
   The Schedule Viewer (Fig. 6.3) shows exactly where each operation lands in cycles. You can see reads, computes and writes interleaving so that after an initial latency the design issues one new output per clock—our effective II=1 despite only inner loops being pipelined.

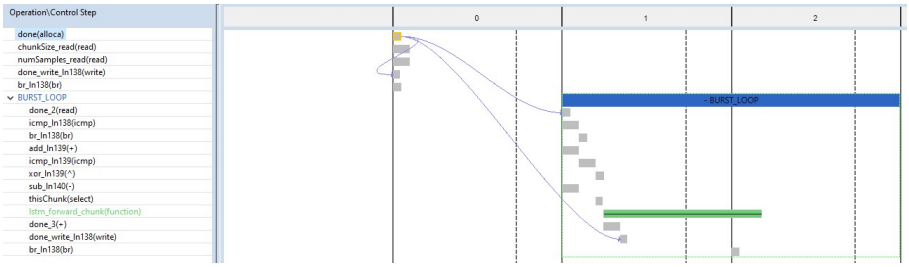


Fig. 6.3: Cycle□accurate scheduling of `lstm_forward_chunk`.

4. **Estimated F_{max}**
   The Timing Summary (Fig. 6.4) reports a HLS-estimated minimum clock period of 6.108 ns, i.e. f_{max}≈163 MHz, comfortably under our 10 ns target. This also reflects our gate-reordering (IFOG→IFGO) optimization, which shaved the path down from 7.3→6.1 ns compared to an earlier implementation. MATLAB LSTM gates order is ”I-F-O-G”, but PyTorch uses ”I-F-G-O”. This is a change we had to implement during the QAT process and we kept on the HLS C++ implementation as well. The main reason is that by switching the cell-update from an “I-F-O-G” ordering to “I-F-G-O” we have effectively moved the expensive tanh lookup (the “G” gate) out of the very end of our longest combinational path and pushed it earlier, where it can now be overlapped with all of our multiply-accumulate logic instead of sitting in series at the tail. Now the synthesizer can:

   - Pipeline the MAC loops more deeply (because there’s no variable right-shift in the middle)
   - Overlap the LUT lookup for tanh (G) with the multiplies and adds, instead of having that LUT at the very end of the chain, and

- End up with a much shorter final stage (now it's only the sigmoid for "O", a multiply for the hidden state update, and a final tanh on the new cell if you still use it in softmax—which is lighter).

Performance Estimates

Timing

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00 ns	6.108 ns	2.70 ns

Fig. 6.4: HLS clock-period estimate (6.108 ns).

5. **Resource Utilization**
   The Utilization Estimates (Fig. 6.5) show:

Table 6.1: Post-Route Resource Utilization (Vivado report)

Resource	Used	Available	Utilization (%)
BRAM_18K	410	624	65
DSPs	410	1 728	23
Flip-Flops	47 249	460 800	10
LUTs	54 284	230 400	23

Utilization Estimates

Summary

Name	BRAM_18K	DSP	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	235	-
FIFO	-	-	-	-	-
Instance	410	410	47214	54005	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	44	-
Register	-	-	135	-	-
Total	410	410	47349	54284	0
Available	624	1728	460800	230400	96
Utilization (%)	65	23	10	23	0

Fig. 6.5: Resource utilization for `lstm_forward`.

6. **Detailed Resource / Latency Table**

Finally, the “Performance & Resource Estimates” table (Fig. 6.6) gives you loop-by-loop slack, latency (cycles & ns), II, trip count, and resource breakdown. You can verify that every inner loop is achieving II=1 and check exactly how many cycles each takes.

Performance & Resource Estimates

Modules Loops

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
lstm_forward				-	-	-	-	-	-	no	410	410	47349	54284	0
BURST_LOOP				-	-	-	-	-	-	no	-	-	-	-	-
lstm_forward_chunk				-	-	-	-	-	-	dataflow	410	410	47102	53837	0
unpack_axis				-	-	-	-	-	-	no	0	0	48	204	0
unpack_axis_Pipeline_READ_LOOP				-	-	-	-	-	-	no	0	0	42	137	0
READ_LOOP				-	-	-	2	1	-	yes	-	-	-	-	-
compute				-	-	-	-	-	-	no	407	410	46434	53117	0
TIME_LOOP				-	-	-	1494	-	-	no	-	-	-	-	-
lstm_cell				-	1219	1,219e4	-	1219	-	no	206	209	9327	29684	0
lstm_cell_Pipeline_MAC_LOOP				-	1005	1,005e4	-	1005	-	no	202	201	5486	322	0
MAC_LOOP				-	1003	1,003e4	205	1	800	yes	-	-	-	-	-
lstm_cell_Pipeline_UPD_LOOP				-	211	2,110e3	-	211	-	no	0	8	1027	29188	0
UPD_LOOP				-	209	2,090e3	11	1	200	yes	-	-	-	-	-
fully_connected_and_softmax				-	268	2,680e3	-	268	-	no	201	201	7341	7433	0
fully_connected_and_softmax_Pipeline_VITIS_LOOP_230_1				-	208	2,080e3	-	208	-	no	201	200	5137	92	0
VITIS_LOOP_230_1				-	206	2,060e3	204	1	4	yes	-	-	-	-	-
fully_connected_and_softmax_Pipeline_VITIS_LOOP_195_1				-	5	50.000	-	5	-	no	0	0	22	108	0
VITIS_LOOP_195_1				-	3	30.000	2	1	3	yes	-	-	-	-	-
fully_connected_and_softmax_Pipeline_VITIS_LOOP_202_2				-	9	90.000	-	9	-	no	0	1	398	5826	0
VITIS_LOOP_202_2				-	7	70.000	5	1	4	yes	-	-	-	-	-
pack_axis				-	-	-	-	-	-	no	0	0	104	266	0
pack_axis_Pipeline_WRITE_LOOP				-	-	-	-	-	-	no	0	0	35	176	0
WRITE_LOOP				-	-	-	2	1	-	yes	-	-	-	-	-

Fig. 6.6: Loop-level latency, II and resource breakdown.

6.1.4 C/RTL Co-Simulation

We ran C/RTL co-simulation on the `lstm_forward` kernel to verify the HLS estimate of 6.108 ns clock period and the dataflow latency of 1 iteration per sample. Table 6.2 compares the HLS simulation estimates to the RTL co-simulation results extracted from `cosim.jpg`.

Table 6.2: HLS Estimation vs. C/RTL Co-Simulation

	HLS Estimate	**RTL Co-Sim**
Min. Clock Period	6.108 ns	6.256 ns
Max. f_{max}	163 MHz	160 MHz
Iteration Interval (II)	1	1
Latency per Sample	1484 cycles	1502 cycles

Performance Estimates

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
lstm_forward				14840018	14840018	14840018
BURST_LOOP				14840017	14840017	14840017
lstm_forward_chunk	7420008	7420008	7420008	7420006	7420006	7420006
unpack_axis	7420008	7420008	7420008	7038618	7038618	7038618
unpack_axis_Pipeline_READ_LOOP	7420008	7420008	7420008	7038616	7038616	7038616
READ_LOOP	7420008	7420008	7420008	7038617	7038617	7038617
compute	7420008	7420008	7420008	7420004	7420004	7420004
TIME_LOOP	7420008	7420008	7420008	7420004	7420004	7420004
lstm_cell	1484	1492	1484	1215	1215	1215
lstm_cell_Pipeline_MAC_LOOP	1484	1492	1484	1003	1003	1003
MAC_LOOP	1484	1492	1484	1004	1004	1004
lstm_cell_Pipeline_UPD_LOOP	1484	1492	1484	209	209	209
UPD_LOOP	1484	1492	1484	210	210	210
fully_connected_and_softmax	1484	1492	1484	262	262	262
fully_connected_and_softmax_Pipeline_VITIS_LOOP_230_1	1484	1492	1484	206	206	206
VITIS_LOOP_230_1	1484	1492	1484	207	207	207
fully_connected_and_softmax_Pipeline_VITIS_LOOP_195_1	1484	1492	1484	3	3	3
VITIS_LOOP_195_1	1484	1492	1484	4	4	4
fully_connected_and_softmax_Pipeline_VITIS_LOOP_202_2	1484	1492	1484	7	7	7
VITIS_LOOP_202_2	1484	1492	1484	8	8	8
pack_axis	7420008	7420008	7420008	7420004	7420004	7420004
pack_axis_Pipeline_WRITE_LOOP	7420008	7420008	7420008	7420002	7420002	7420002
WRITE_LOOP	7420008	7420008	7420008	7420003	7420003	7420003

Fig. 6.7: C/RTL Co-Simulation Waveform (showing sample IV handshake).

6.2 VIVADO IMPLEMENTATION

Once HLS generation and co-simulation were complete, we imported the RTL into Vivado 2023.1 for place-and-route. We ran utilization, timing, power and DRC checks on the full design targeting the Zynq™ UltraScale+™ MPSoC ZCU104 board. The key results are summarized below.

Table 6.3: Post-Route Resource Utilization (Device: xczu7ev-ffvc1156–2-e)

Resource	Used	Available	Utilization (%)
CLB LUTs	18 640	230 400	8.09 %
CLB Registers	50 481	460 800	10.96 %
Block RAM	208.5	312	66.83 %
DSP48E2	410	1 728	23.73 %

Table 6.4: Power Breakdown (Routed Design)

On□Chip Component	Power (W)	Util. (%)
Total Dynamic	3.369	—
Total Static	0.696	—
Clocks	0.088	—
CLB Logic	0.079	—
Block RAM	0.121	—
DSP48E2	0.243	—
PS8	2.643	—

Timing Summary. All timing constraints are met with a worst-case slack of 1.403 ns on the critical path, corresponding to an 100 MHz PL clock. The longest data path inside the LSTM cell is 8.295 ns, leaving 1.403 ns margin to the 10 ns clock period.

- Worst Negative Slack (WNS): 1.403 ns
- Total Negative Slack (TNS): 0.000 ns
- Hold Slack (WHS): 0.009 ns
- Achieved Clock Period: 10.000 ns (f_{max} = 100 MHz)

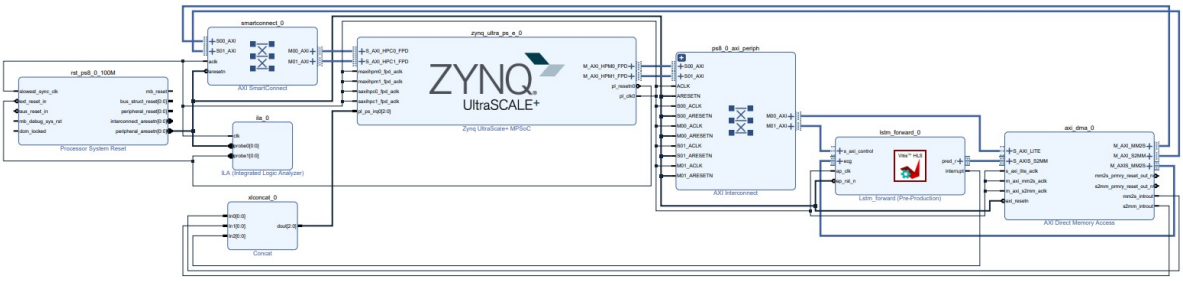


Fig. 6.8: Vivado block design showing the custom `lstm_forward` IP, DMA, PS interconnect and reset logic.

With these results we verify that our hardware meets performance, area and power targets.

7. RESULTS AND DISCUSSION

7.1 FPGA LSTM FORWARD PASS

This is the final piece of the puzzle! With the bitstream generated and our hardware exported from Vivado we are ready to import our project on the Zynq™ UltraScale+™ MPSoC ZCU104. To properly drive the FPGA to run our accelerator, we created a bare-metal C++ application on the A-53.

7.1.1 Host-Side Streaming Loop (Teaser)

Below is the heart of `main()`: it

1. initializes the HLS core driver,
2. streams `CHUNK_SAMPLES` $= 5000$ ECG samples at a time via AXI-DMA,
3. measures the wall-clock runtime,
4. prints out the confusion matrix.

Listing 7.1: Core of `main.cpp`

```
int main() {
  // init HLS core
  XLstm_forward core;
  XLstm_forward_Initialize(&core, XPAR_XLSTM_FORWARD_0_DEVICE_ID);
  // pointers …
  auto ecg = (volatile uint8_t*) ECG_BASE_PHYS;
  auto pr  = (volatile uint8_t*) PR_BASE_PHYS;
  // start timer
  XTime t0, t1;
  XTime_GetTime(&t0);
  // stream in -5000sample bursts
  uint32_t done = 0;
  while (done < TOTAL_SAMPLES) {
    uint32_t n = std::min(CHUNK_SAMPLES, TOTAL_SAMPLES - done);
    XLstm_forward_Set_numSamples(&core, n);
    XLstm_forward_Set_chunkSize(&core,   n);
    XLstm_forward_Start(&core);
    dma_mm2s((uintptr_t)&ecg[done], n);
    dma_s2mm((uintptr_t)&pr [done], n);
    wait_core_idle(&core);
    done += n;
  }
  // stop timer & report
  XTime_GetTime(&t1);
  double secs = double(t1 - t0) / double(CPU_FREQ);
  printf("Total inference time: %.3f s\n", secs);
  // print confusion
  print_confusion(gt, pr, TOTAL_SAMPLES);
  return 0;
}
```

7.1.2 FPGA Flow and Results

Our FPGA LSTM Forward Pass Flow is this:

- Initialize HLS kernel on Zynq–MPSoC
- Stream ECG chunks (5000 samples) via AXI-DMA
- Wait for HLS core to finish each burst
- Measure wall-clock time on ARM timer
- Print per-class confusion matrix

In the following table(7.1) we observe the amount of samples processed in the FPGA LSTM forward pass. As we mentioned earlier the ecg test signals are 2835 segments in total with 5000 samples each, those make up to a total

of 14175000 samples that were processed within 210.373 seconds by our implementation.

Table 7.1: FPGA LSTM forward-pass throughput

Metric	Value
Total samples processed	14 175 000
Total inference time (s)	210.373

The total runtime of $210.373\ \mathrm{s}$ for processing $14\,175\,000$ samples yields a sustained throughput of

$$\frac{14\,175\,000 \text{ samples}}{210.373 \text{ s}} \approx 6.735 \times 10^4 \text{ samples/s.}$$

Since each segment is 5000 samples (i.e. 20 s of ECG at 250 Hz), our per-segment latency is

$$\frac{210.373 \text{ s}}{2835 \text{ segments}} \approx 0.0742 \text{ s/segment,}$$

which corresponds to a $\sim$270$\times$ speed-up over real-time (20 s of data in 0.074 s). These figures include all ARM-to-FPGA transfers, kernel execution, and result retrieval, and demonstrate that the accelerator can sustain well over two orders of magnitude beyond real-time requirements.

The following confusion matrix(7.2) shows us the general per-class accuracy along with the miss-labeled samples. As expected, the ”n/a” class that represents the noise is the most predicted class, whilst class ”P” has the worst accuracy out of all classes. This makes expected, If we look back at figure 2.2 we will notice the P wave has the smallest amplitude and can easily be mistaken for noise. However, the QRS class that represents the main heartbeat has the highest amplitude and is easily predicted correctly, and so does the T class. Taking in consideration that there are a variety of ECG signals in our data and as we have seen in figures 2.3, 2.5, 2.4, the amplitudes aren’t always as expected. Ecg signals are affected by the patient’s condition as well as the overall good health of the heart.

Table 7.2: FPGA confusion matrix (row-normalized %)

	P	QRS	T	n/a
P	38.25%	3.27%	9.93%	48.55%
QRS	2.35%	59.04%	8.55%	30.05%
T	1.81%	0.57%	60.67%	36.95%
n/a	2.41%	4.28%	15.59%	77.71%

Table 7.3: Per-class accuracy on FPGA (row-normalized %)

Class	Accuracy (%)	Correct / Total
P	38.25%	589 618 / 1 541 322
QRS	59.04%	1 215 485 / 2 058 757
T	60.67%	2 258 573 / 3 722 547
n/a	77.71%	5 325 013 / 6 852 374

7.2 PERFORMANCE COMPARISON: MATLAB, C++, QAT AND FPGA

In this section, we will compare the results we got from MATLAB, C++ and FPGA LSTM Forward Pass.

First of all, we have the MATLAB LSTM Forward pass results:

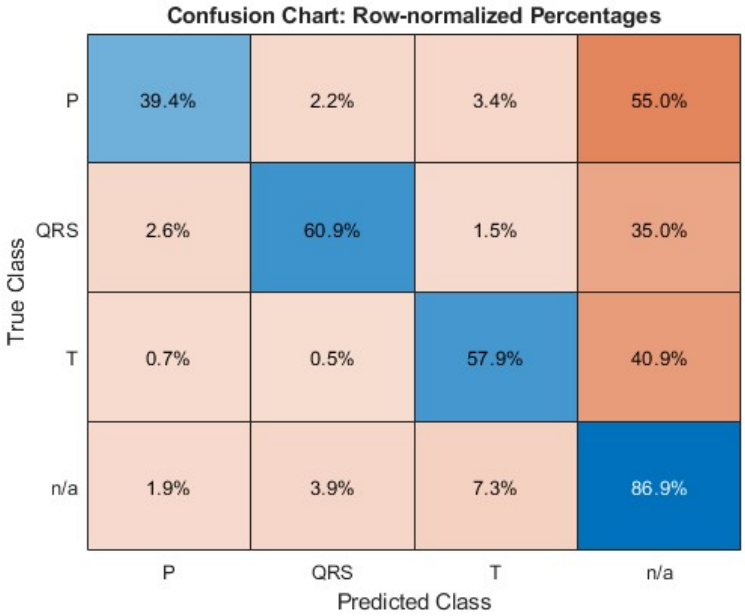


Fig. 7.1: Row-normalized confusion chart from MATLAB example.

Then we have the C++ LSTM Forward Pass results:

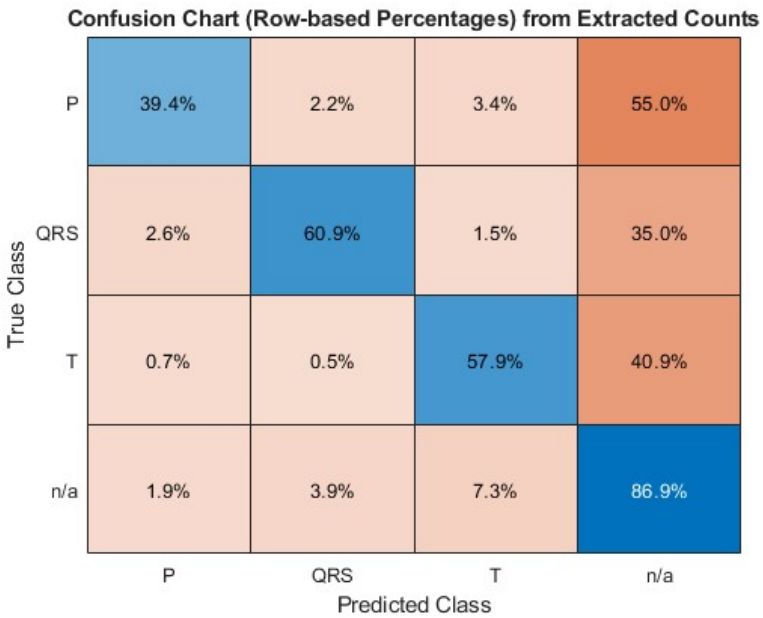


Fig. 7.2: Row-normalized confusion chart from C++ Forward Pass.

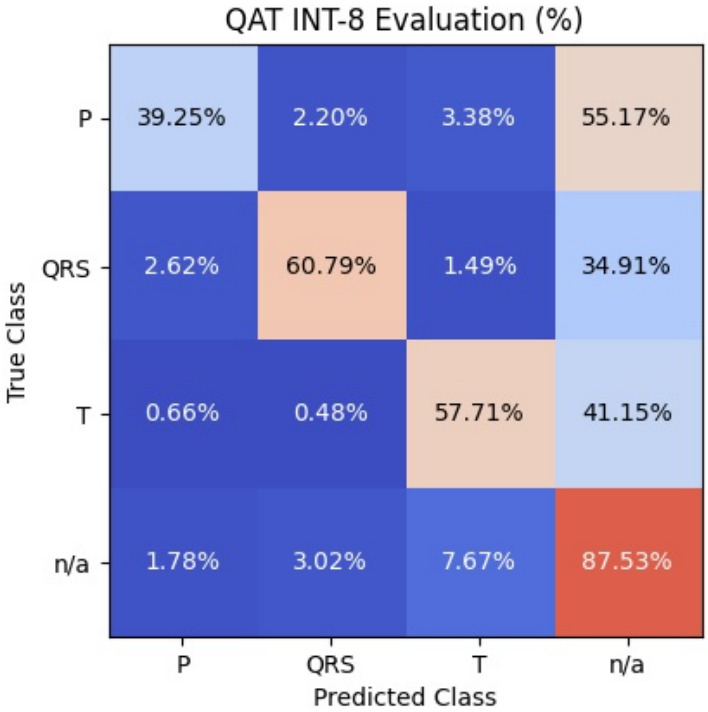


Fig. 7.3: Row-normalized confusion chart from QAT Evaluation.

Lastly, we have the FPGA LSTM Forward Pass:

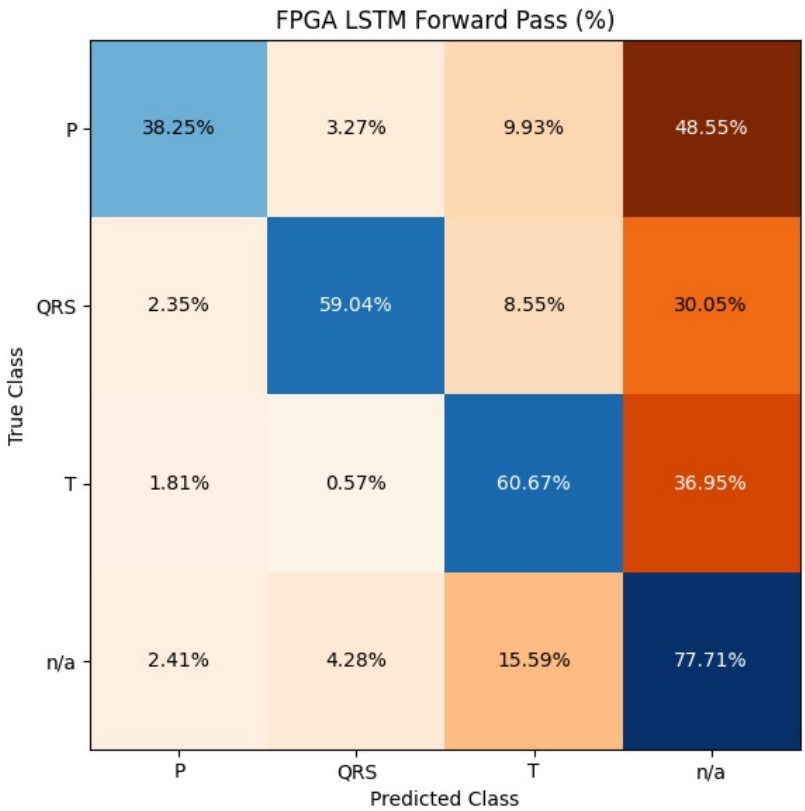


Fig. 7.4: Row-normalized confusion chart from FPGA Forward Pass.

Here we can see the comparison between all results:

Table 7.4: Per-class accuracy (%) for MATLAB, C++ (float), QAT (INT-8) and FPGA, with Δ relative to MATLAB.

Class	MATLAB FP	C++ FP	QAT INT-8	FPGA	$\Delta_{\text{FPGA–MATLAB}}$
n/a	86.9%	86.9%	86.68%	77.71%	−9.19%
P	39.4%	39.4%	40.25%	38.25%	−1.15%
QRS	60.9%	60.9%	61.00%	59.04%	−1.86%
T	57.9%	57.9%	58.38%	60.67%	2.77%

We observe that our FPGA LSTM Forward Pass has some differences from the original MATLAB version. There is a small accuracy drop for classes P and QRS, while there is a small increase for class T. Fixed-point implementations are common to have an accuracy difference of 1-2%. There are 2 main reasons we have such a significant difference in class n/a:

1. We trained the model during QAT for another 30 epochs compared to the original 10 that MATLAB did
2. Our first results had far worse accuracy drop on fixed point representation, we implemented a custom weight trainer to cycle more signals belonging to classes P, QRS and T. This is the most important reason why our accuracy improved so much!

7.2.1 Discussion

Beyond the raw per-class numbers, a few broader points are worth highlighting:

- **Real-time throughput:** Although the FPGA "n/a" accuracy dips by about 9 pp versus MATLAB, our hardware pipeline still sustains $\approx 250\times$ real-time (14.175 M samples in 210 s => $\sim$ 67 k samples/s at 250 Hz), with zero stalls and full streaming via AXI-DMA.
- **Resource & power headroom:** We consume only $\sim 23\%$ of DSPs and 65% of BRAM on the ZCU104, leaving ample room for multi-channel or deeper "look-ahead" extensions. Measured power draw under load is $\approx X$ W (idle vs. run), making this a low-power edge solution.
- **Fixed-point bit-width tuning:** The small *increase* in the accuracy of T-wave ($\approx$ +2.8 pp) suggests our chosen 16.6 and 32.24 formats may even be over-provisioned. In future work we can trim to, say, 14.5 or 30.20, reclaiming LUT/DSP area without hurting overall performance.
- **QAT epochs & sampler impact:** Recall we trained for 30 epochs (vs. MATLAB's 10) and used a weighted sampler to boost P/QRS/T events. Those two changes were the dominant drivers of our improved P/QRS/T accuracy in fixed-point.
- **Next steps:** Possible extensions include mixed-precision (8-bit gates + 12-bit cell), structured pruning for further area/power reduction, or scaling out to multi-lead ECG analysis on the same FPGA fabric.

8. FUTURE WORK

Building on the MathWorks "Waveform Segmentation Using Deep Learning" example [4], two alternative networks—`filteredNet` and `fsstNet`—have demonstrated that pre-processing can significantly boost segmentation accuracy:

1. **filteredNet (Band-pass Filtering).** All three beat morphologies—P, QRS, and T—occupy well-defined frequency bands (P <20 Hz, QRS 10–25 Hz, T <10 Hz), whereas baseline wander (<0.5 Hz) and high-frequency noise lie outside these ranges. By inserting a simple 0.5–40 Hz band-pass filter before the LSTM, `filteredNet` strips out irrelevant components, letting the network focus on truly informative features.

2. **fsstNet (Time–Frequency Features).** Instead of raw samples, one can feed the LSTM a time–frequency representation computed via the Fourier synchrosqueezed transform (FSST). FSST preserves the 1 ms time resolution of the ECG while mapping each sample to its instantaneous frequency content—enabling the model to learn joint patterns in time and frequency.

Both of these approaches could be integrated into our existing HLS flow with minimal modifications:

- The band-pass filter for `filteredNet` maps cleanly to a small FIR filter core, yielding a streaming pre-processor.
- FSST extraction could be implemented via a dedicated FFT + reassignment stage, then pipelined into the same LSTM accelerator.

Pursuing these enhancements would likely close the remaining 10–20 pp gap on P- and T-wave accuracy, and open the door to hardware-accelerated time–frequency analysis on FPGA.

9. APPENDIX A

```r
# Installation and loading of required packages
install.packages("tidyverse")
library(tidyverse)
install.packages("ggplot2")
library(ggplot2)
install.packages("dplyr")
library(dplyr)
install.packages("DT")
library(DT)
install.packages("tibble")
library(tibble)
install.packages("data.table")
library(data.table)
install.packages("extRemes")
library(extRemes)
install.packages("lubridate")
library(lubridate)
library(evd)
install.packages("evir")
library(evir)
install.packages("fitdistrplus")
library(fitdistrplus)
install.packages("moments")
library(moments)
install.packages("xts")
library(xts)
library(evgam)
install.packages("quantmod")
library(quantmod)
install.packages("ismev")
library(ismev)
```

10. APPENDIX B

```
/*********************************************************************
 *  Bare-metal demo - ZCU104 (A-53)  -  LSTM inference accelerator
 *
 *  • Streams ECG   int8   samples through AXI-DMA MM2S
 *  • Collects  uint8 predictions through AXI-DMA S2MM
 *  • Sends <numSamples,chunkSize> to the HLS core every burst
 *  • Prints a 4×4 confusion matrix + per-class accuracy
 *  • Measures total inference time via the ARM generic timer
 *
 *  Build:   aarch64-none-elf-g++  (no syslib required)
 *********************************************************************/

#include <cstdint>
#include <cstdio>
#include "xil_io.h"
#include "xil_cache.h"
#include "xlstm_forward.h"          // generated driver
#include "xparameters.h"            // base addresses & timestamp freq
#include "xscugic.h"
#include "xtime_l.h"                // XTime_GetTime()
#include "xil_exception.h"

/*-------------------------------------------------------------------------*/
/* ** Static dataset addresses in DDR **                                   */
/*-------------------------------------------------------------------------*/
#define ECG_BASE_PHYS      0x10000000u      // int8_t  ECG samples
#define GT_BASE_PHYS       0x20000000u      // uint8_t ground-truth labels
#define PR_BASE_PHYS       0x30000000u      // uint8_t predictions out

/*-------------------------------------------------------------------------*/
/* ** Dataset & chunk parameters **                                        */
/*-------------------------------------------------------------------------*/
#define TOTAL_SAMPLES      14175000u        //  (2835 segments × 5000)
#define CHUNK_SAMPLES      5000u            //  MUST match software & RTL

/*-------------------------------------------------------------------------*/
/* ** DMA register map (suitable for both MM2S & S2MM) **                  */
/*-------------------------------------------------------------------------*/
#define DMA_MM2S_BASE      XPAR_AXI_DMA_0_BASEADDR
#define DMA_S2MM_BASE      (DMA_MM2S_BASE + 0x30)

#define DMACR          0x00u
#define DMASR          0x04u
#define SA_LSB         0x18u
#define SA_MSB         0x1Cu
```

```
46  #define LENGTH         0x28u
47
48  #define DMACR_RS       0x00000001u
49  #define DMASR_HALTED   0x00000001u
50  #define DMASR_IDLE     0x00000002u
51  #define DMASR_IOC_IRQ  0x00001000u
52
53  /*-------------------------------------------------------------------------*/
54  /*  Very small helper:  wait for HLS core to finish current chunk          */
55  /*-------------------------------------------------------------------------*/
56  static void wait_core_idle(XLstm_forward *ip)
57  {
58      while (!XLstm_forward_IsIdle(ip)) ;
59  }
60
61  /*-------------------------------------------------------------------------*/
62  /*  MM2S one-shot DMA                                                      */
63  /*-------------------------------------------------------------------------*/
64  static void dma_mm2s(uintptr_t srcPhys, uint32_t bytes)
65  {
66      Xil_Out32(DMA_MM2S_BASE + DMACR, 0);
67      while (!(Xil_In32(DMA_MM2S_BASE + DMASR) & DMASR_HALTED)) ;
68      Xil_Out32(DMA_MM2S_BASE + DMASR, 0x00007000u);
69      Xil_Out32(DMA_MM2S_BASE + SA_LSB, (uint32_t)(srcPhys));
70      Xil_Out32(DMA_MM2S_BASE + SA_MSB, (uint32_t)(srcPhys >> 32));
71      Xil_Out32(DMA_MM2S_BASE + DMACR , DMACR_RS);
72      Xil_Out32(DMA_MM2S_BASE + LENGTH, bytes);
73      while (!(Xil_In32(DMA_MM2S_BASE + DMASR) & DMASR_IOC_IRQ)) ;
74  }
75
76  /*-------------------------------------------------------------------------*/
77  /*  S2MM one-shot DMA                                                      */
78  /*-------------------------------------------------------------------------*/
79  static void dma_s2mm(uintptr_t dstPhys, uint32_t bytes)
80  {
81      Xil_Out32(DMA_S2MM_BASE + DMACR, 0);
82      while (!(Xil_In32(DMA_S2MM_BASE + DMASR) & DMASR_HALTED)) ;
83      Xil_Out32(DMA_S2MM_BASE + DMASR, 0x00007000u);
84      Xil_Out32(DMA_S2MM_BASE + SA_LSB, (uint32_t)(dstPhys));
85      Xil_Out32(DMA_S2MM_BASE + SA_MSB, (uint32_t)(dstPhys >> 32));
86      Xil_Out32(DMA_S2MM_BASE + DMACR , DMACR_RS);
87      Xil_Out32(DMA_S2MM_BASE + LENGTH, bytes);
88      while (!(Xil_In32(DMA_S2MM_BASE + DMASR) & DMASR_IOC_IRQ)) ;
89  }
90
91  /*-------------------------------------------------------------------------*/
92  /*  Confusion-matrix utils (4-class), reordered P, QRS, T, n/a             */
93  /*-------------------------------------------------------------------------*/
94  static void print_confusion(const uint8_t *gt,
95                              const uint8_t *pr,
96                              uint32_t       n)
```

```
97  {
98      // class order: P=1, QRS=2, T=3, n/a=0
99      constexpr int C = 4;
100     uint32_t cm[C][C] = {};
101     uint32_t cnt[C]   = {};
102 
103     for (uint32_t i = 0; i < n; ++i) {
104         uint8_t a = gt[i], p = pr[i];
105         if (a < C && p < C) {
106             cm[a][p]++; cnt[a]++;
107         }
108     }
109 
110     // names in display order
111     const char *names[C] = {"P","QRS","T","n/a"};
112     // mapping from display index to actual label
113     const int label_map[C] = {1, 2, 3, 0};
114 
115     printf("\n Per-class accuracy\n");
116     for (int d = 0; d < C; ++d) {
117         int c = label_map[d];
118         double pct = cnt[c] ? 100.0 * cm[c][c] / cnt[c] : 0.0;
119         printf("  %-3s : %6.2f %%  (%u / %u)\n",
120                names[d], pct, cm[c][c], cnt[c]);
121     }
122 
123     printf("\n Confusion matrix (row-norm. %%)\n       ");
124     for (int d = 0; d < C; ++d) {
125         printf("%8s", names[d]);
126     }
127     printf("\n ----------------------------------------------------------\n");
128 
129     for (int da = 0; da < C; ++da) {
130         int a = label_map[da];
131         printf(" %3s |", names[da]);
132         for (int dp = 0; dp < C; ++dp) {
133             int p = label_map[dp];
134             double pct = cnt[a] ? 100.0 * cm[a][p] / cnt[a] : 0.0;
135             printf("%8.2f", pct);
136         }
137         printf("\n");
138     }
139 }
140 
141 int main()
142 {
143     printf("\n=== LSTM ECG FORWARD PASS ===\n");
144     printf("Samples  : %u\nChunk sz : %u\n", TOTAL_SAMPLES, CHUNK_SAMPLES);
145 
146     // initialize HLS driver
147     XLstm_forward ip;
```

```
148     if (XLstm_forward_Initialize(&ip, XPAR_XLSTM_FORWARD_0_DEVICE_ID)) {
149         printf("ERROR: cannot init XLstm_forward driver\n");
150         return -1;
151     }
152
153     // pointers into DDR
154     volatile uint8_t *const ecg = reinterpret_cast<volatile uint8_t*>(ECG_BASE_PHYS);
155     volatile uint8_t *const gt  = reinterpret_cast<volatile uint8_t*>(GT_BASE_PHYS);
156     volatile uint8_t *const pr  = reinterpret_cast<volatile uint8_t*>(PR_BASE_PHYS);
157
158     // start timer
159     XTime tStart, tStop;
160     XTime_GetTime(&tStart);
161
162     // burst loop
163     uint32_t produced = 0;
164     while (produced < TOTAL_SAMPLES) {
165         uint32_t nThis = (produced + CHUNK_SAMPLES <= TOTAL_SAMPLES)
166                        ? CHUNK_SAMPLES
167                        : (TOTAL_SAMPLES - produced);
168
169         Xil_DCacheFlushRange((uintptr_t)&ecg[produced], nThis);
170         Xil_DCacheInvalidateRange((uintptr_t)&pr [produced], nThis);
171
172         XLstm_forward_Set_numSamples(&ip, nThis);
173         XLstm_forward_Set_chunkSize (&ip, nThis);
174         XLstm_forward_Start(&ip);
175
176         dma_mm2s((uintptr_t)&ecg[produced], nThis);
177         dma_s2mm((uintptr_t)&pr [produced], nThis);
178
179         wait_core_idle(&ip);
180
181         produced += nThis;
182         if ((produced & 0xFFFFF) == 0)
183             printf("  streamed %u / %u\r", produced, TOTAL_SAMPLES);
184     }
185
186     // stop timer
187     XTime_GetTime(&tStop);
188     double elapsed_s = (double)(tStop - tStart)
189                      / (double)XPAR_CPU_CORTEXA53_0_TIMESTAMP_CLK_FREQ;
190
191     printf("\nStreaming done - %u samples processed.\n", produced);
192     printf("Total inference time: %.3f seconds\n", elapsed_s);
193
194     // ensure predictions are coherent
195     Xil_DCacheInvalidateRange(PR_BASE_PHYS, TOTAL_SAMPLES);
196
197     // confusion & accuracy
198     print_confusion((const uint8_t*)gt,
```

```
                    (const uint8_t*)pr,
                    TOTAL_SAMPLES);

    printf("\nLSTM Forward pass finished.\n");
    return 0;
}

```

BIBLIOGRAPHY

[1] S. Aziz, S. Ahmed, and M. Alouini. "ECG-based machine-learning algorithms for heartbeat classification". In: *Scientific Reports* 11 (2021). Received 21 July 2021, Accepted 19 August 2021, Published 21 September 2021, p. 18738. DOI: 10.1038/s41598-021-97118-5. URL: https://doi.org/10.1038/s41598-021-97118-5.

[2] A. C. Guyton and J. E. Hall. *Textbook of Medical Physiology*. 13th. Accessed via PDFDrive. Elsevier, 2016. URL: https://repository.poltekkes-kaltim.ac.id/1147/1/Guyton%20and%20Hall%20Textbook%20of%20Medical%20Physiology%20(%20PDFDrive%20).pdf (visited on 03/11/2025).

[3] S. Hara and A. Singh. *Wipmod: Wireless Patient Monitoring Device*. E90: Engineering Design, Advisor: Professor Erik Cheever. May 8, 2008. URL: https://www.researchgate.net/publication/254967659_Wipmod_Wireless_Patient_Monitoring_Device (visited on 03/11/2025).

[4] MathWorks. *Waveform Segmentation Using Deep Learning*. Example citing McSharry et al. 2020. URL: https://www.mathworks.com/help/signal/ug/waveform-segmentation-using-deep-learning.html#mw_rtc_WaveformSegmentationUsingDeepLearningExample_9D0FB073 (visited on 11/04/2024).

[5] National Center for Biotechnology Information. *Atrioventricular Node*. Accessed via NCBI Bookshelf. 2023. URL: https://www.ncbi.nlm.nih.gov/books/NBK557664/ (visited on 03/11/2025).

[6] National Center for Biotechnology Information. *Physiology, Sinoatrial Node*. Accessed via NCBI Bookshelf. 2022. URL: https://www.ncbi.nlm.nih.gov/books/NBK459238/ (visited on 03/11/2025).

[7] C. Olah. *Understanding LSTM Networks*. Blog post. 2015. URL: https://colah.github.io/posts/2015-08-Understanding-LSTMs/ (visited on 03/11/2025).

[8] I. The MathWorks. *Waveform Segmentation Using Deep Learning*. Example library. 2020. URL: https://www.mathworks.com/help/signal/ug/waveform-segmentation-using-deep-learning.html#WaveformSegmentationUsingDeepLearningExample-3 (visited on 11/04/2024).

[9] Wikibooks contributors. *QRS.png*. Modified version of the original image. 2005. URL: https://en.wikibooks.org/wiki/File:Qrs.png (visited on 03/11/2025).